

✓ Natural Language Processing (NLP)

```
import nltk
#nltk.download()
```

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

Word Tokenizer

```
import nltk
from nltk.tokenize import word_tokenize

text = "Welcome to the Python Programming at Indeed Inspiring Infote
print(word_tokenize(text))
```

[Show hidden output](#)

Sentence Tokenizer

```
from nltk.tokenize import sent_tokenize
text = "Hello Everyone. Welcome to the Python Programming"
print(sent_tokenize(text))
```

```
['Hello Everyone.', 'Welcome to the Python Programming']
```

Stemming

```
from nltk.stem import PorterStemmer
# words = ['Wait', 'Waiting', 'Waited', 'Waits']
words = ['clean', 'cleaning', 'cleans', 'cleaned']
ps = PorterStemmer()
for w in words:
    words=ps.stem(w)
    print(words)
```

```
clean
clean
clean
clean
```

✓ Lemmatization

Stemming Code:

```
import nltk
from nltk.stem.porter import PorterStemmer
porter_stemmer= PorterStemmer()
text = "studies studying floors cry"
tokenization= nltk.word_tokenize(text)
for w in tokenization:
    print('Stemming for ', w, 'is', porter_stemmer.stem(w))
```

```
Stemming for studies is studi
Stemming for studying is studi
Stemming for floors is floor
Stemming for cry is cri
```

Lemmatization Code:

```
import nltk
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
True
```

```
import nltk
from nltk.stem import WordNetLemmatizer
Wordnet_lemmatizer=WordNetLemmatizer()
text = "studies study floors cry"
tokenization= nltk.word_tokenize(text)
for w in tokenization:
    print('Lemma for ', w,'is',Wordnet_lemmatizer.lemmatize(w))
```

```
Lemma for  studies is study
Lemma for  study is study
Lemma for  floors is floor
Lemma for  cry is cry
```

✓ NLTK stopwords

```
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords

data = 'AI was introduced in the year 1956 butit gained popularity'
stopwords = set(stopwords.words('english'))
words=word_tokenize(data)
wordsFiltered =[]
```

```
for w in words:
    if w not in stopwords:
        wordsFiltered.append(w)
```

```
print(wordsFiltered)
```

```
['AI', 'introduced', 'year', '1956', 'butit', 'gained', 'popularity']
```

```
print(len(stopwords))
print(stopwords)
```

```
198
{'isn't', 're', 'no', 'own', 'weren', 'y', 'have', "you've", 'because'}
```

✓ Text Analytics

1. Extract Sample document and apply following document preprocessing methods: Tokenization, POS Tagging, stop words removal, Stemming and Lemmatization.
2. Create representation of document by calculating Term Frequency and Inverse Document Frequency

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag
```

```
# Example document
document = "This is an example document that we will use to demonst
```

```
# Tokenization
tokens = word_tokenize(document)
```

tokens

```
['This',
 'is',
 'an',
 'example',
 'document',
 'that',
 'we',
 'will',
 'use',
 'to',
 'demonstrate',
 'document',
 'preprocessing',
 '.']
```

```
import nltk
nltk.download('averaged_perceptron_tagger')
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]   date!
True
```

```
import nltk
```

```
# Download required resources
```

```
nltk.download('punkt')
```

```
nltk.download('averaged_perceptron_tagger_eng')
```

```
from nltk.tokenize import word_tokenize
```

```
from nltk.tag import pos_tag
```

```
pos_tags = pos_tag(tokens)
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
```

```
[nltk_data]   Package punkt is already up-to-date!
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
```

```
[nltk_data]   /root/nltk_data...
```

```
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-t
```

```
[nltk_data]   date!
```

```
pos_tags
# DT for determiner , NN for noun , VBD for past tense word , IN for
#VBZ: verb, 3rd person singular present tense (e.g. "runs")
# VB: verb, base form (e.g. "run")
# PRP: personal pronoun (e.g. "he", "she", "it", "they")
# MD: modal verb (e.g. "can", "should", "will")
# TO: to (e.g. "to run", "to go")
# Here are some additional tags that you may find useful:
# NN: noun, singular or mass (e.g. "cat", "dog", "water")
# NNS: noun, plural (e.g. "cats", "dogs")
# JJ: adjective (e.g. "happy", "blue")
# RB: adverb (e.g. "quickly", "very")
# IN: preposition or subordinating conjunction(e.g. "in", "on", "be
```

```
[('This', 'DT'),
 ('is', 'VBZ'),
 ('an', 'DT'),
 ('example', 'NN'),
 ('document', 'NN'),
 ('that', 'IN'),
 ('we', 'PRP'),
 ('will', 'MD'),
 ('use', 'VB'),
 ('to', 'TO'),
 ('demonstrate', 'VB'),
 ('document', 'NN'),
 ('preprocessing', 'NN'),
 ('.', '.')]

```

```
#Stopwords removal
stop_words= set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if not word.lower() in s
```

```
#Stemming
ps=PorterStemmer()
stemmed_tokens= [ps.stem(word) for word in filtered_tokens]
```

```
#Lemmatization
wnl=WordNetLemmatizer()
lemmatized_tokens = [wnl.lemmatize(word) for word in filtered_token
```

```
# Print the results
print("Tokens: ",tokens)
# print("POS tags: ", pos_tags)
print("Filtered tokens: ", filtered_tokens)
print("Stemmed tokens: ", stemmed_tokens)
print("Lemmatized tokens: ", lemmatized_tokens)
#NLTK is capable of performing all the document preprocessing methc
```

```
Tokens: ['This', 'is', 'an', 'example', 'document', 'that', 'we', '
Filtered tokens: ['example', 'document', 'use', 'demonstrate', 'doc
Stemmed tokens: ['exampl', 'document', 'use', 'demonstr', 'document
Lemmatized tokens: ['example', 'document', 'use', 'demonstrate', 'c
```

Text Analytics

Create representation of document by calculating Term Frequency and Inverse Document Frequency.

```
import math
from collections import Counter
```

```
# Sample corpus of documents
corpus=[
    'The quick brown fox jumps over the lazy dog',
    'The brown fox is quick',
    'The lazy dog is sleeping'
]
```

```
# Tokenize the documents
tokenized_docs = [doc.lower().split() for doc in corpus]
```

```
# Count the term frequency for each document
tf_docs = [Counter(tokens) for tokens in tokenized_docs]
```

```
# Calculate the inverse document frequency for each term
n_docs = len(corpus)
idf = {}
for tokens in tokenized_docs:
    for token in set(tokens):
        idf[token] = idf.get(token, 0) + 1
for token in idf:
    idf[token] = math.log(n_docs / idf[token])
```

```
# Calculate the TF-IDF weights for each document
tfidf_docs = []
for tf_doc in tf_docs:
    tfidf_doc = {}
    for token, freq in tf_doc.items():
        tfidf_doc[token] = freq * idf[token]
    tfidf_docs.append(tfidf_doc)
```

```
# Print the resulting TF-IDF representation for each document
for i, tfidf_doc in enumerate(tfidf_docs):
    print(f"Document {i+1}: {tfidf_doc}")
```

```
Document 1: {'the': 0.0, 'quick': 0.4054651081081644, 'brown': 0.4054651081081644}
Document 2: {'the': 0.0, 'brown': 0.4054651081081644, 'fox': 0.4054651081081644}
Document 3: {'the': 0.0, 'lazy': 0.4054651081081644, 'dog': 0.4054651081081644}
```

This code uses the Counter class from the collections module to count the term frequency for each document. It then calculates the inverse document frequency by iterating over the tokenized documents and keeping track of the number of documents that each term appears in. Finally, it multiplies the term frequency of each term in each document by its corresponding inverse document frequency to get the TF-IDF weight for each term in each document. The resulting TF-IDF representation for each document is printed to the console.

